

CORTEX

Master Roadmap & Team Execution Plan

How to Build the MVP, Start to Finish, as 4 People Working in Parallel

Purpose of this document. The Ground Source of Truth (Ch. 1–20) says *what* CORTEX is. This document says *who builds what, in what order, and how the four of you stay unblocked from each other for five weeks*. Read this first on Day 1; use it every Monday and Friday as the standup script.

ET AI Hackathon 2026 — Problem Statement #8

AI for Industrial Knowledge Intelligence: Unified Asset & Operations Brain

Team Insomniacs

Bhavishy Agrawal · Suparn Agrawal

Anshika Jha · Anarghya Das

July 2026

Companion document to *CORTEX MVP — Ground Source of Truth*. Derived from and consistent with Ch. 20 (Future Enterprise Roadmap is a different chapter — this is the *build* roadmap, not the post-hackathon one).

Contents

1	How We Approach This Build	5
1.1	The one idea that makes 4 people $\times$ 5 weeks work	5
1.2	Five working principles	5
1.3	Definition of done for the whole project	6
2	Team & Track Ownership	7
2.1	Why four tracks, not four features	7
2.2	What each track is explicitly NOT responsible for	8
2.3	Weekly rhythm	8
3	Contracts to Freeze in Week 1	9
3.1	Contract 1 — The REST/SSE/WS API surface	9
3.2	Contract 2 — The shared graph-access function	9
3.3	Contract 3 — The streamed event schema	9
4	Phase-by-Phase Roadmap (Weeks 1–5)	11
4.1	How to read this chapter	11
4.2	Week 1 — Foundations (maximum parallelism)	11
4.3	Week 2 — Core Pipelines	12
4.4	Week 3 — Advanced Retrieval + Compositional Reasoning	12
4.5	Week 4 — Integration Hardening	13
4.6	Week 5 — Hardening, Evaluation, Rehearsal	13
5	Cross-Track Dependency Map	15
5.1	Who is waiting on whom, and when it resolves	15
5.2	The two seams every track is built against	15
6	Risk Register & Fallback Ownership	17
6.1	Principle	17
6.2	Rehearse the fallback, don't just code it	19
7	Definition of Done, Per Subsystem	20
8	Parking Lot — What We Are Deliberately Not Building	22

List of Tables

2.1	Track ownership. Names are a suggested default split; swap freely, but keep exactly one clear owner per track so there is always one throat to choke per deliverable. . .	7
2.2	Negative scope per track — as important as the positive scope for keeping tracks from colliding	8
3.1	The three Week-1 contracts and who depends on each	10
4.1	Week 1 per-track deliverables	11
4.2	Week 2 per-track deliverables	12
4.3	Week 3 per-track deliverables	13
4.4	Week 4 per-track deliverables	13
4.5	Week 5 activities — no longer split by track, the whole team converges	14
5.1	Cross-track dependency map, in dependency order	15
6.1	Risk register with explicit fallback ownership	18
7.1	Acceptance targets — the numbers Week 5 must actually measure, not estimate . .	21

List of Figures

- 4.1 The five-week arc — each week's sync point is the entry condition for the next . . . 11

Chapter 1

How We Approach This Build

1.1 The one idea that makes 4 people $\times$ 5 weeks work

Core strategy

Define three contracts by the end of Week 1 — the API surface, the shared graph-access function signature, and the streaming event schema — and then let all four tracks build against those contracts independently. Nobody waits for anybody else’s internal implementation to be finished; everybody waits only for the *shape* of the interface, which is fixed on Day 5 and never renegotiated except at a sync point.

If this project fails, it will not fail because someone wrote bad code. It will fail because two people silently assumed different shapes for the same interface and only discovered it in Week 4. Every rule in this chapter exists to prevent that one failure mode.

1.2 Five working principles

1. **Contracts before code.** The request/response shape of every endpoint ([Chapter 3](#)) is written down and agreed before anyone implements the logic behind it. A contract change after Week 1 requires all four people to acknowledge it in the same conversation, not a silent Slack message to one person.
2. **Build against a stub, not a wait.** If Track C needs Track B’s retrieval function before it exists, Track C writes a stub that returns fixture data in the exact shape of the real contract, and swaps it out later without changing its own code. This is how Weeks 1–2 achieve full parallelism.
3. **Vertical slices over horizontal layers.** Each week ends with something that actually runs end-to-end, however roughly ([Chapter 4](#)), rather than four people each perfecting one layer in isolation until Week 5.
4. **Freeze scope, not quality.** Week 5 adds zero new features ([Section 4.6](#)). Anything not named in the Ground Source of Truth chapters 3–18 is written into the roadmap-appendix parking lot ([Chapter 8](#)) and not built this cycle.
5. **Every stage degrades, nothing crashes.** Each track owns building the fallback for its own stage ([Chapter 6](#)) as a first-class deliverable, not a stretch goal bolted on in Week 5.

1.3 Definition of done for the whole project

The project is done when the exact demo script in Ground Source of Truth §18.2 runs, unrehearsed-but-rehearsable, end to end, against the real 100–300 document corpus, in under eight minutes, with every number quoted on stage pulled live from the evaluation dashboard. Everything in this roadmap points at that one sentence.

Chapter 2

Team & Track Ownership

2.1 Why four tracks, not four features

The team splits along the system’s own architectural seams (Fabric API’s internal layers and the two surrounding processes), not along arbitrary feature slices, because the seams are exactly where the contracts in [Chapter 3](#) already isolate one person’s work from another’s.

Track	Suggested owner	Owns
A — DATA & INGESTION	Bhavishy Agrawal	Postgres/Neo4j schemas, Docling pipeline, entity+spec extraction, entity resolution, P&ID VLM path, knowledge-write stage
B — RETRIEVAL & KNOWLEDGE GRAPH	Suparn Agrawal	Neo4j/Qdrant setup, query classification, context assembly, seed expansion, adaptive traversal, structural enrichment, fusion/re-rank, citation resolution
C — AGENTS & REASONING	Anshika Jha	LangGraph supervisor, CORTEX Asset (incl. CGFR), CORTEX Diagnose, CORTEX Comply, model-routing for agent-side calls
D — FRONTEND, AUTH & INFRA	Anarghya Das	Next.js frontend, graph explorer, streaming UI, auth/RBAC, Docker Compose, deployment, demo rehearsal logistics

Table 2.1: Track ownership. Names are a suggested default split; swap freely, but keep exactly one clear owner per track so there is always one throat to choke per deliverable.

One owner does not mean one worker. Any track owner can and should pull in another team member for a day when their own track is blocked on a sync point (e.g. Track D can help Track A test ingestion edge cases while waiting for Track B's real retrieval function in Week 2). Ownership means *accountable for the deliverable existing by the sync date, not the only person allowed to touch the code*.

2.2 What each track is explicitly NOT responsible for

Track	Explicitly out of this track's scope
A	Anything query-time (retrieval, generation, agents) — Track A's world ends the moment a fact is written to Postgres/Neo4j/Qdrant.
B	Agent orchestration and prompts — Track B exposes <code>retrieve()</code> and the graph traversal internals; it does not decide when an agent is invoked.
C	Chunking, fusion weights, or traversal internals — Track C calls <code>context_graph_query</code> and <code>retrieve()</code> as black boxes and never reaches into Neo4j/Qdrant directly.
D	Any business logic — the frontend renders whatever Fabric API streams; it does not compute confidence scores, fusion weights, or agent routing decisions.

Table 2.2: Negative scope per track — as important as the positive scope for keeping tracks from colliding

2.3 Weekly rhythm

- **Monday (start of week):** 20-minute standup against this document — each track states this week's deliverable from [Chapter 4](#) and any dependency it is waiting on.
- **Wednesday (midweek check):** Async check-in; anyone blocked flags it immediately rather than at Friday sync, since a 2-day slip on one track's contract is a 5-week-project risk.
- **Friday (sync point):** The specific, testable sync-point criterion for that week ([Chapter 4](#)) is demonstrated live by whoever owns it — not described, demonstrated. If it cannot be demonstrated, it is not done, regardless of how much code exists.

Chapter 3

Contracts to Freeze in Week 1

Why this chapter exists

Everything in Chapter 4 after Week 1 assumes these three shapes are already fixed. If any of these three change after the Week 1 sync point, treat it as a whole-team event, not a one-line Slack message — every track that consumes the changed contract needs to re-confirm it still works for them.

3.1 Contract 1 — The REST/SSE/WS API surface

The complete endpoint list in Ground Source of Truth Ch. 12 (`/api/documents`, `/api/query`, `/api/entities/*`, `/api/agents/*`, `/ws/copilot/{session_id}`) and the SSE event schema (`token`, `citation`, `agent_trigger`, `done`, plus CGFR's `agent_step` sub-events) are written down verbatim and shared before anyone writes a handler body. **Owner:** Track D drafts it from the Ground Source of Truth, Track B and C confirm it covers everything they need to emit.

3.2 Contract 2 — The shared graph-access function

[language=Python, caption=The one function every retrieval consumer and every agent worker calls — this signature does not change after Week 1] `async def context_graph_query(tag : str, query : str, depth : str = "auto", include_analogues : bool = False,) -> GraphContext : """'shallow' : fixed - depthtraversal, max_depth = 2(100ms) - -fastpath.'deep' : fullfive - phasecontext - awarepipeline, max_depth = 5(300-500ms).!auto' : shallowforAsset(history)/Comply, deepforDiagnose.'"""`
Owner: Track B owns what happens inside every mode; Track C only ever calls this function and never reaches into Neo4j directly. Track C stubs this function's return shape in Week 1-2 and swaps in Track B's real implementation in Week 3 without changing a single call site.

3.3 Contract 3 — The streamed event schema

[language=json, caption=Event shapes every consumer of the streaming API is built against from Week 1] `event: token data: "text": "..."` `event: citation data: "doc_id" : "...", "passage_id" : "...", "page" : 0` `event: agent_trigger data: "worker" : "asset|diagnose|comply", "job_id" : "..."` `event: agent_step data: "step" : "constraint|candidate|compatibility_check|recommendation|plan|hypothesis|critique"` `done data: "answer_id" : "..."` **Owner:** Track D builds the frontend against mocked events matching this shape starting Week 1; Track B and Track C emit real events matching it starting Week 2-3.

Contract	Frozen by	Consumed by
API surface (§3.1)	End of Week 1	All four tracks
context_graph_query signature (§3.2)	End of Week 1	Track C (caller), Track B (implementer)
Streamed event schema (§3.3)	End of Week 1	Track D (renderer), Track B/C (emitters)

Table 3.1: The three Week-1 contracts and who depends on each

Chapter 4

Phase-by-Phase Roadmap (Weeks 1–5)

4.1 How to read this chapter

Each week has: a table of per-track deliverables, an explicit **sync-point criterion** (the one thing that must be demonstrable live, not described), and a short note on what each track is unblocked to start *because* of that sync point. The five weeks trace one continuous arc: *skeleton* → *core pipelines* → *advanced reasoning* → *integration hardening* → *freeze and rehearse*.

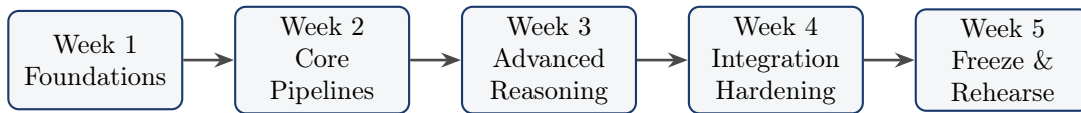


Figure 4.1: The five-week arc — each week’s sync point is the entry condition for the next

4.2 Week 1 — Foundations (maximum parallelism)

The goal of Week 1 is not features; it is *unblocking*. Every track can start on Day 1 with zero dependency on another track’s code, only on the contracts in [Chapter 3](#).

Track	Week 1 deliverable
A	Postgres schema live (<code>document_registry</code> , <code>facts</code> , <code>entity_registry</code> , <code>entity_aliases</code> , <code>users</code> , <code>audit_log</code>) with migrations 001/002 applied; Docling integration skeleton; the upload-to-registry write path working end to end.
B	Neo4j + Qdrant containers up; ontology node/edge model coded (labels, relational and spec-type relationship types); chunking policy implemented (structure-aware, 512-token cap); a basic vector pathway with no fusion yet.
C	LangGraph supervisor skeleton routing to three stub workers; fast-tier and frontier-tier model SDK wrappers callable end to end (even against a placeholder prompt).
D	Full Docker Compose stack (all 4 deployable units) boots together; Next.js scaffold with <code>/login</code> ; JWT issuance and <code>require_role</code> guard; 4 seeded demo accounts.

Table 4.1: Week 1 per-track deliverables

Sync point — end of Week 1

`docker compose up` brings up all four services together, and a file uploaded via `POST /api/documents` lands as a `document_registry` row. The three contracts in [Chapter 3](#) are frozen and written

down somewhere every team member can see them.

4.3 Week 2 — Core Pipelines

With contracts frozen, every track now builds its first real, non-stubbed pipeline segment.

Track	Week 2 deliverable
A	Fast-tier structured extraction live against the extended predicate set (relational + spec-type); 2.5-stage entity resolution (exact/fuzzy/embedding); Neo4j <code>MERGE</code> write path; a P&ID VLM stub returning placeholder adjacency.
B	Query classification (regex/keyword pre-filter + LLM fallback); <code>assemble_context</code> implemented, including the <code>focused_tag</code> plumbing that Track D exposes from the graph explorer; Strategy A (direct) and Strategy B (embedding-nearest) seed expansion.
C	CORTEX Asset capability 1–2 (history/QA, forward-risk framing) working end to end against Track B’s early/stub graph data; CORTEX Diagnose’s plan→execute skeleton; CORTEX Comply’s gap-check Cypher query.
D	Graph Explorer (Cytoscape.js) with node click opening the side panel; CopilotChat SSE client wired against Track B’s early output or mocked events; route guards wired to the real endpoints (not just <code>/login</code>).

Table 4.2: Week 2 per-track deliverables

Sync point — end of Week 2

A ~10-document fixture corpus ingests end to end (upload → parse → extract → resolve → graph + vectors written), and the very first real Copilot query returns a citation-tagged answer — however rough the citations or however narrow the graph context.

Highest-risk dependency, check explicitly at this sync point. Track C’s CGFR quality in Week 3 depends entirely on Track A having populated `hasSpec/fitsRange/` etc. on the spare-parts catalog. If this is not visibly true by the end of Week 2, escalate it immediately — do not wait for the Week 3 sync point to discover CGFR has nothing to reason over.

4.4 Week 3 — Advanced Retrieval + Compositional Reasoning

This is the week the system’s actual differentiators get built: adaptive traversal and CGFR.

Track	Week 3 deliverable
A	Full 100–300 document corpus ingested; alias table matured from real documents; contradicting-fact and dedup edge cases handled per the fact/trust model.
B	Adaptive traversal (best-first, relevance-decayed, node-budget bounded); structural enrichment (multi-seed shortest paths + degree-based hub detection); fusion weights per query type; local cross-encoder re-rank; citation-resolution regeneration loop.
C	CGFR tools (<code>get_entity_specs</code> , <code>find_by_type</code> , <code>compare_specs</code>) and the full four-step chain (Constraint Extraction → Candidate Gathering → Compatibility Filtering → Justified Selection) wired inside CORTEX Asset, calling Track B’s real <code>retrieve()</code> and Track A’s real spec-fact coverage.
D	ReasoningOverlay with Replay control; the <code>ComparisonTable</code> component; the compliance view; a mobile-responsive pass.

Table 4.3: Week 3 per-track deliverables

Sync point — end of Week 3

The first end-to-end CGFR answer — “which funnel should I use for pipe P-301?” — renders in the real UI with a real, data-backed comparison table, not mocked data.

4.5 Week 4 — Integration Hardening

Every stubbed error path and every stubbed fallback from Weeks 1–3 becomes real. This is the week that turns “it works on the happy path” into “it survives a live demo.”

Track	Week 4 deliverable
A	Ch. 8-style error/degradation paths implemented for ingestion (<code>parse_failed</code> , retry-once extraction, unresolved-mention logging); <code>audit_log</code> writes from the ingestion side.
B	Shallow↔full traversal mode switch tuned against the latency targets (Ground Source of Truth ??); a performance pass against the phase-by-phase budget table.
C	Autonomy-tier wiring (Tier-2 confirm/dismiss, coordinated with Track D); <code>max_iterations</code> caps enforced on all three workers; sequence-numbered replay logging.
D	Full API surface wired end to end (not just query/entities); both SSE and WS transports live; auth edge cases (expired token mid-stream); the uniform error envelope on every endpoint.

Table 4.4: Week 4 per-track deliverables

Sync point — end of Week 4

The full Playwright/Cypress end-to-end script passes, and measured latency is checked against the Chapter 4 §4.5 targets end to end, not just per-component.

4.6 Week 5 — Hardening, Evaluation, Rehearsal

No new features this week. If a team member finds themselves wanting to add anything not already named in Ground Source of Truth Ch. 3–18, it goes into Chapter 8, not into the

codebase. Week 5 is fixes, evaluation, and rehearsal only.

All four tracks, converged	Activity
Evaluation	Build the acceptance-target test sets (retrieval precision set, citation spot-check set, CGFR test set, compliance gap set); split across all four people; cross-review each other's correct/incorrect judgements.
Bug bash	Full-corpus bug bash; deliberately trigger each of the three fallback paths (P&ID pre-load, structural-enrichment drop, CGFR zero-candidate) at least once and confirm each degrades gracefully instead of crashing.
Rehearsal	Full demo script rehearsed end to end at least three times against the real demo corpus, timed with a stopwatch; every team member able to deliver every speaking point, with the compositional-reasoning beat rehearsed at least once more given it is the newest, most judge-differentiating part of the script.
Freeze	Final full-corpus re-ingestion; final latency pass; scope frozen — fixes only from this point forward.

Table 4.5: Week 5 activities — no longer split by track, the whole team converges

Final acceptance criterion

Every number the team plans to say out loud to a judge has actually been measured on the actual demo corpus, by a team member, this week — not estimated, not carried over from a smaller test, not asserted from memory.

Chapter 5

Cross-Track Dependency Map

5.1 Who is waiting on whom, and when it resolves

Dependency	Detail	Resolves by
C → B	CORTEX Asset/Diagnose/Comply all call <code>context_graph_query</code> and <code>retrieve()</code> ; lock the signature Week 1, stub it Week 1–2, swap in the real implementation Week 3 without changing the call sites.	Week 3
D → B/C	Frontend’s streaming components are built against the SSE/WS event schema (Chapter 3 , Contract 3); lock it Week 1 so Track D can build against mocked events while B/C assemble the real pipeline.	Week 1 (contract), Week 3 (real events)
C → A	CGFR’s quality depends entirely on Track A’s spec-predicate extraction coverage on the spare-parts catalog; this is the single highest-risk cross-track dependency — checked explicitly at the Week 2 sync point (Section 4.3).	Week 2 (checked), Week 3 (consumed)
B → A	Adaptive traversal and seed expansion need real graph data, not fixture-only data, to tune relevance thresholds meaningfully.	Week 3 (full corpus from A)
D → C	Tier-2 confirm/dismiss UI wiring needs the autonomy-tier contract from Track C (what a “proposed” action payload looks like).	Week 4

Table 5.1: Cross-track dependency map, in dependency order

5.2 The two seams every track is built against

Why only two seams matter

The `context_graph_query` signature and the streamed-event structure are the two points every other component is built against. Instability in either has system-wide impact — treat any proposed change to either after Week 1 as a whole-team decision, not a one-track decision.

Chapter 6

Risk Register & Fallback Ownership

6.1 Principle

Every pipeline stage has a defined fallback, and that fallback is owned by the same track that owns the stage — it is not a Week-5 afterthought, it is part of that track’s Week 1–4 deliverable.

Risk	Fall A back to 2-3 pre-processed drawings with graph edges pre-loaded via a seed script	Drop B the phase, proceed on adaptive-traversal results alone	Conf A + demo C the corpus's seeded spare-parts catalog in advance; if it happens live anyway, the explicit "no com
P&ID live VLM extraction slow/unreliable	Week 4		
Structural enrichment exceeds its latency budget	Week 4		
CGFR candidate gathering finds zero candidates on the live corpus	Week 3-4		

6.2 Rehearse the fallback, don't just code it

A fallback path that has never actually been triggered on purpose is not a tested fallback — it is an assumption. Week 5's bug bash ([Section 4.6](#)) requires each of the three major fallback paths above to be deliberately triggered at least once and confirmed to degrade gracefully, not merely reviewed in code.

Chapter 7

Definition of Done, Per Subsystem

Metric
Knowledge coverage
Citation accuracy (20 spot-checked answers)
Retrieval precision@5 (15–20 hand-written queries, incl. ≥ 3 implicit-context and ≥ 2 cross-domain-bridge)
Agent success rate (scripted invocations)
Compositional accuracy (5–8 hand-built CGFR queries)
Compliance detection (5 deliberately seeded gaps)

Table 7.1: Acceptance targets — the numbers Week 5 must actually measure, not estimate

Not in scope for acceptance testing: load testing beyond the P95 latency harness, security penetration testing, multi-region failover.

Chapter 8

Parking Lot — What We Are Deliberately Not Building

The rule

If, while building any part of this roadmap, a team member wants to add a service, database, agent, or framework not already named in the Ground Source of Truth (Ch. 3–18), the correct action is to write it here and keep building the MVP as scoped — not to build it now.

- Kafka, Kubernetes/K3s, a bitemporal claim store, ontology versioning, a typed **Specification** node label.
- A DuckDB/ClickHouse analytical mirror; RT-DETR+SAM2 line tracing for P&IDs.
- Three-tier autonomy with an OPA policy engine; Keycloak/OIDC/mTLS; OT/historian integration.
- Offline-first PWA with CRDT sync; Microsoft GraphRAG-style offline community summarisation; Neo4j GDS Enterprise.
- Fine-tuned models; a published-benchmark-style evaluation with inter-annotator agreement studies.
- A dedicated material-compatibility lookup database; a company-specific **selectionRule** relationship type.
- A fourth agent worker; a LiteLLM abstraction layer or a third/fourth model tier.

If the project continues past the hackathon, Ground Source of Truth Ch. 20 (*Future Enterprise Roadmap*) gives the trigger condition and recommended sequencing for each of these — that chapter, not this one, is where post-MVP investment gets planned.